minor-6-bash-version-guard

MINOR-6 fix evidence — `install-dev-tools.sh` bash>=4.0 guard

Revision: 1 **Last modified:** 2026-08-18T00:00:00Z

Source finding: `.superpowers/sdd/task-review-457cca4-a7e55f9-report.md` MINOR-6 (commit `c7dfdde`) — `declare -A RESULT` requires bash >= 4.0; macOS ships bash 3.2 by default; no early runtime-version check existed, so a `--check` invocation on stock macOS would fail with a cryptic `declare: -A: invalid option` instead of an actionable message.

Fix applied

`scripts/install-dev-tools.sh`:

1. Header comment gained a “Runtime requirement — bash >= 4.0 (§11.4.35 consumer-DATA declaration)” paragraph naming the exact cause (associative-array status table), the exact failure mode on macOS, and the exact remediation (`brew install bash` + explicit re-invocation via `$(brew --prefix bash)/bin/bash`).
2. Immediately after `set -uo pipefail` (before any other statement, including the argument-parsing loop), a guard checks the REAL, authoritative `BASH_VERSINFO` array — never inferred from `uname -s` or OS family, since a Homebrew/upgraded bash on macOS is a legitimate PASS (§11.4.201 guard-asserts-real-condition: the condition is “can this interpreter run an associative array”, not “is this macOS”). On violation it prints the exact cause + the exact fix and exits 2 (the script’s own documented “invocation error” exit class — never conflated with the FAILED/exit-1 class reserved for genuine install errors).

§11.4.6 anti-bluff verification — golden-good / golden-bad

Golden-good (real host, bash 5.2.37, `--check` mode unaffected)

```
$ bash --version | head -1
GNU bash, version 5.2.37(1)-release (x86_64-alt-linux-gnu)

$ bash scripts/install-dev-tools.sh --check --tool wrk >/tmp/
idt_check.log 2>&1
$ echo "real exit code (no pipe): $?"
real exit code (no pipe): 0

$ tail -3 /tmp/idt_check.log
Summary (check):
  wrk: ALREADY-PRESENT: /home/milosvasic/bin/wrk (wrk 4.2.0
[epoll] Copyright (C) 2012 Will Glozer)
========================================
```

`bash -n scripts/install-dev-tools.sh` — clean, before AND after the edit.

Golden-bad (guard logic proven against a genuinely bash-3.x-shaped condition)

An actual bash 3.2 interpreter is not available on this host (verified: only bash 5.2.37 is installed; `apt-cache`/package search for a legacy bash package returns nothing on this ALT Linux host, and the real `BASH_VERSINFO` array is a bash-internal read-only value the running interpreter sets itself — it cannot be overridden from inside a running bash 5.2 process to fake a bash-3 identity). Per §11.4.6 this limitation is stated honestly rather than silently assumed away.

The guard's LOGIC (not the shipped variable name — an isolated probe using a simulated array of the same shape, so the comparison itself is proven under the golden-bad condition) was extracted and run standalone:

```
$ cat /tmp/guard_probe.sh
#!/usr/bin/env bash
set -uo pipefail
BASH_VERSINFO_SIM=(3 2 57 1 release x86_64-apple-darwin)
if [ -z "${BASH_VERSINFO_SIM:-}" ] || [ "${BASH_VERSINFO_SIM[0]}"
-lt 4 ]; then
  echo "install-dev-tools: bash >= 4.0 required (associative
arrays); got 3.2.57(1)-release (simulated)." >&2
  echo "  macOS ships bash 3.2 by default. Fix: brew install bash,
then re-run with:" >&2
```

```
  echo "    \$(brew --prefix bash)/bin/bash $0 $*" >&2
  exit 2
fi
echo "SHOULD NOT REACH HERE"

$ bash /tmp/guard_probe.sh --check
install-dev-tools: bash >= 4.0 required (associative arrays); got
3.2.57(1)-release (simulated).
  macOS ships bash 3.2 by default. Fix: brew install bash, then
re-run with:
    $(brew --prefix bash)/bin/bash /tmp/guard_probe.sh --check
$ echo "exit=$?"
exit=2
```

The probe never reaches `"SHOULD NOT REACH HERE"` — the version-gate comparison (`[ "${ARR[0]}" -lt 4 ]`) correctly trips on `3` and the script exits 2 with the actionable message BEFORE any tool-install logic runs. This is the exact numeric-comparison pattern shipped in `scripts/install-dev-tools.sh`'s guard (only the array identifier differs: the real interpreter's own `BASH_VERSINFO` vs. the probe's `BASH_VERSINFO_SIM`, substituted only because the genuine array cannot be overridden from a running bash 5.2 process).

Honest boundary (§11.4.6)

This fix is verified by (a) the real host's bash 5.2 golden-good path (unaffected — exit 0, unchanged behavior) and (b) an isolated golden-bad probe proving the exact numeric-comparison + exit-2 + actionable-message logic trips correctly on a bash-3-shaped version array. It has NOT been independently verified on a real, unmodified macOS host running a genuine bash 3.2 binary — no such host was available in this session. This is an honest §11.4.3 gap, not a silent claim of full cross-platform proof; the header comment added to the script states this limitation explicitly and names the exact remediation path for an operator who does hit it.